CSCI 2406: Topic 06

Data Transfer and Addressing

Memory operands, load/store widths, addressing modes, alignment, and data traversal

Computer Organization & Architecture | AArch64 Reference

Topic 6 Outline

Topic Objective

Explain how AArch64 transfers values between registers and memory and how an instruction forms the effective address for a memory access.

- **Section 1**: Memory addresses and load/store semantics
- **Section 2**: Transfer width and extension
- **Section 3**: AArch64 addressing modes
- **Section 4**: Arrays, pointers, and strings
- **Section 5**: Alignment, pair transfers, and program-memory context
- **Section 6**: Summary and self-test

Section 01

Memory Addresses and Load/Store Semantics

AArch64 Is a Load/Store Architecture

Separation of Memory Access and Computation

Arithmetic and logical instructions operate on register values. Dedicated load and store instructions transfer values between registers and memory.

```
ldr x1, [x0]
add x1, x1, #5
str x1, [x0]
```

- `ldr` reads a value from memory into a register.
- `add` changes the register value.
- `str` writes the register value back to memory.

Memory Is Byte Addressed

- Each memory address identifies one byte.
- Multi-byte values occupy consecutive addresses.
- The instruction determines how many bytes are transferred.
- The base address for a load/store is held in a 64-bit base register or in sp.

Example

A 64-bit value beginning at `0x1000` occupies byte addresses `0x1000` through `0x1007`.

Address	Byte
`0x1000`	byte 0
`0x1001`	byte 1
`0x1002`	byte 2
⋮	
`0x1007`	byte 7

Bracket Notation Means Memory Access

Register Operand

```
add x0, x1, x2
```

- Uses values already in registers.
- No data-memory operand appears.

Memory Operand

```
ldr x0, [x1]
```

- `x1` contains an address.
- Brackets mean access memory at that address.

Do Not Confuse Address and Data

The value in `x1` is the address. The bytes stored at that address are the data transferred by the load or store.

Loading Data with `ldr`

64-Bit Load

```
ldr x0, [x1]
```

- The effective address is the 64-bit value in `x1`.
- Eight bytes are read from memory.
- The 64-bit value is written into `x0`.

32-Bit Load

`ldr w0, [x1]` reads four bytes into `w0`. Any write to `w0` clears bits [63:32] of the corresponding `x0` register.

Storing Data with `str`

64-Bit Store

```
str x0, [x1]
```

- The effective address is the value in `x1`.
- The eight-byte value in `x0` is written to memory.
- The source register is not changed by the store.

Narrow Stores

`strb` writes the low 8 bits of a register, and `strh` writes the low 16 bits. `str w0, [x1]` writes the low 32 bits.

Section 02

Transfer Width and Extension

Load Width Determines Bytes Read

Instruction	Bytes Read	Destination Result
`ldrb w0, [x1]`	1 byte	Zero-extended to 32 bits in `w0`
`ldrh w0, [x1]`	2 bytes	Zero-extended to 32 bits in `w0`
`ldr w0, [x1]`	4 bytes	Written to `w0`; upper half of `x0` becomes zero
`ldr x0, [x1]`	8 bytes	Full 64-bit value written to `x0`

Width Comes from the Instruction

A memory address does not carry a type. The load/store instruction determines the number of bytes transferred.

Signed Loads Preserve Negative Values

Sign Extension

A signed load copies the sign bit of the narrow source into the new upper bits so the two's-complement value is preserved.

Instruction	Source Width	Result
`ldrsb x0, [x1]`	8 bits	Sign-extended to 64 bits
`ldrsh x0, [x1]`	16 bits	Sign-extended to 64 bits
`ldrsw x0, [x1]`	32 bits	Sign-extended to 64 bits

Example

If memory contains the byte `0xff`, `ldrb` interprets the transferred bits as 255 after zero extension, while `ldrsb x0, [x1]` produces the 64-bit representation of -1.

Load Width and Extension Example

Assume memory at `0x2000` contains the byte `0x80`.

Unsigned Byte Load

`ldrb w0, [x1]` `w0 = 0x00000080`

- Decimal value: 128
- Upper bits filled with zeros

Signed Byte Load

`ldrsb x0, [x1]` `x0 = 0xffffffffffffff80`

- Decimal value: -128
- Sign bit replicated upward

Section 03

AArch64 Addressing Modes

The Effective Address

Definition

The effective address is the memory address computed for a particular load or store instruction.

- A base register supplies the starting address.
- An optional immediate or register offset can modify that address.
- Some addressing forms also write an updated address back to the base register.

Base Register

AArch64 memory operands normally use a 64-bit general-purpose register or sp as the base. A w register is not used as the base address register.

Base Register Addressing

Syntax

```
ldr x0, [x1]
```

- Effective address = `x1`.
- `x1` is unchanged.
- This corresponds closely to dereferencing a pointer already holding the required address.

C Analogy

If `x1` represents pointer `p`, then the memory operand `[x1]` corresponds conceptually to `*p`.

Immediate Offset Addressing

Example

```
ldr x0, [x1, #16]
```

- Effective address $= \texttt{x1} + 16$.
- The base register `x1` is unchanged.
- Useful for structure fields, stack-frame locations, and fixed-position array elements.

Encoding Detail

A64 has more than one load/store immediate encoding. Common unsigned offsets are scaled by transfer size; unscaled forms use a smaller signed byte offset.

Register Offset Addressing

Unscaled Register Offset

`ldr x0, [x1, x2]` Effective address $= \texttt{x1} + \texttt{x2}$.

Scaled Index for 64-Bit Elements

`ldr x0, [x1, x2, lsl #3]` Effective address $= \texttt{x1} + (\texttt{x2} \times 8)$.

Register-offset forms are well suited to array indexing because the index can be scaled to the element size.

Using a 32-Bit Index Register

AArch64 register-offset addressing can extend a 32-bit index while forming a 64-bit address.

Unsigned 32-Bit Index

```
ldr x0, [x1, w2, uxtw #3]
```

- `w2` is zero-extended to address width.
- `lsl #3` scales the index by 8 bytes.
- This is useful when the source-language array index is a 32-bit unsigned value.

Signed Index

`sxtw` can be used instead when the 32-bit index must be sign-extended before address formation.

Pre-Indexed Addressing

Example

```
ldr x0, [x1, #8]!
```

1. Compute `x1 + 8`.
2. Use the updated address for the memory access.
3. Write the updated address back into `x1`.

Syntax Marker

The exclamation mark indicates writeback. In A64, pre-indexed writeback uses an immediate offset, not a general-purpose register offset.

Post-Indexed Addressing

Example

```
ldr x0, [x1], #8
```

1. Access memory using the original value of `x1`.
2. After the access, compute `x1 + 8`.
3. Write the updated address back into `x1`.

Common Use

Post-indexing is useful when processing the current element and then advancing a pointer to the next element.

Addressing Mode Comparison

Form	Example	Access Address	Base After
Base	`ldr x0, [x1]`	`x1`	unchanged
Immediate offset	`ldr x0, [x1, #16]`	`x1 + 16`	unchanged
Register offset	`ldr x0, [x1, x2]`	`x1 + x2`	unchanged
Pre-index	`ldr x0, [x1, #8]!`	`x1 + 8`	`x1 + 8`
Post-index	`ldr x0, [x1], #8`	`x1`	`x1 + 8`

Key Distinction

Offset addressing computes an access address without changing the base register; pre-indexed and post-indexed forms also write an updated address back to the base.

Section 04

Arrays, Pointers, and Strings

Array Address Calculation

For an array with element size S, element `A[i]` begins at:

$$\text{address}(A[i]) = \text{base}(A) + i \times S$$

Example: 64-Bit Array

If `x1` contains the array base and `x2` contains index `i`:

```
ldr x0, [x1, x2, lsl #3]
```

- `lsl #3` multiplies the index by $2^3 = 8$.
- The computed byte offset selects the requested 64-bit element.

Traversing an Array with a Pointer

Assume `x0` points to the current 64-bit array element.

```
ldr x1, [x0], #8
```

- The load copies the current 64-bit element into `x1`.
- The memory access uses the original address in `x0`.
- After the load, post-index writeback advances `x0` by 8 bytes.
- `x0` therefore points to the next array element.

Pointer Traversal

Post-index addressing is useful when each iteration processes the current element and then advances to the next one.

Traversing a Null-Terminated String

A C-style string is a byte array ending with a zero byte.

```
loop:
    ldrb w1, [x0], #1
    cbz w1, done
    // process character in w1
    b loop
done:
```

- `ldrb` reads one byte.
- Post-indexing advances the pointer by one byte.
- `cbz` ends the loop when the null terminator is loaded.

Obtaining the Address of a Static Object

Teaching Convenience

Assemblers commonly accept pseudo-instructions such as: `ldr x0, =array`

- This asks the assembler/linker toolchain to arrange for the address of `array` to be placed in `x0`.
- It is not one universal A64 machine encoding called "load address."
- The assembler may use a literal load or another instruction sequence depending on the target and relocation model.

Later Detail

Position-independent code often forms addresses using instructions such as `adrp` plus `add`; exact sequences are discussed when relocation and linking are introduced.

Section 05

Alignment, Pair Transfers, and Program-Memory Context

Natural Alignment

Natural Alignment Rule

An n-byte object is naturally aligned when its starting address is a multiple of n.

- 2-byte halfword: address divisible by 2.
- 4-byte word: address divisible by 4.
- 8-byte doubleword: address divisible by 8.

AArch64 Qualification

Many ordinary AArch64 loads and stores to normal memory permit unaligned addresses, but alignment can affect performance and some access classes have stricter requirements. Device, atomic, and exclusive accesses require additional care.

Load and Store Pair: `ldp` and `stp`

Store Pair

```
stp x0, x1, [x2]
```

- Stores two register values.
- Addresses are adjacent according to operand width.

Load Pair

```
ldp x0, x1, [x2]
```

- Loads two register values.
- Common in stack-frame save/restore code.

Important Qualification

A pair transfer is one architectural instruction, but it should not be assumed to be atomic or to have a universal timing advantage on every processor.

Pair Transfers and the Stack

Function Entry Pattern

```
stp x29, x30, [sp, #-16]!
```

- Pre-indexing first subtracts 16 from sp.
- The two 64-bit values are then stored at the new stack location.

Function Exit Pattern

```
ldp x29, x30, [sp], #16
```

Post-indexing loads the pair and then restores sp by adding 16. Procedure conventions are developed fully in Topic 8.

Program Memory Layout Is OS and Toolchain Context

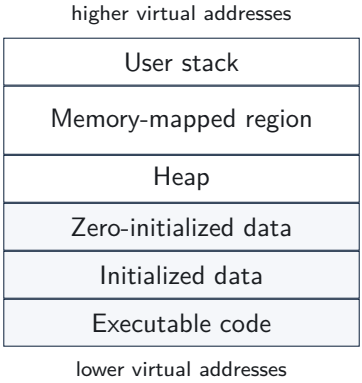
ISA vs. Process Layout

AArch64 defines load/store instruction behavior. The operating system, executable format, linker, and ABI determine how a particular process is arranged in virtual memory.

- ELF is a common executable/object format on Unix-like AArch64 systems.
- ELF **sections** such as `.text`, `.data`, and `.bss` organize linker/object-file content.
- ELF **program segments** describe what the loader maps into a process address space.
- These ideas provide context for where instructions and data come from, but they are not A64 addressing modes.

Typical Linux User-Space Layout

- Executable code and static data are mapped from the program image.
- The heap is used for dynamic allocation.
- Shared libraries and other mappings occupy memory-mapped regions.
- The user stack holds stack frames and local process state.
- ASLR and other OS choices mean exact addresses and ordering are not fixed by AArch64.



ELF Sections and Loadable Segments

Sections: Linker View

- `.text`: executable instructions
- `.rodata`: read-only constants
- `.data`: initialized writable data
- `.bss`: zero-initialized storage

Segments: Loader View

- Program headers describe loadable regions.
- A segment may contain several sections.
- Permissions can distinguish executable, read-only, and writable mappings.

Important Detail

`.bss` usually contributes memory size without storing an equivalent block of zero bytes in the executable file; the loader arranges zero-initialized memory.

Section 06

Summary and Self-Test

Topic 6 Summary

- AArch64 uses explicit load/store instructions to move values between registers and memory.
- The instruction determines transfer width and whether a narrow loaded value is zero-extended or sign-extended.
- Effective addresses can use a base register, immediate offset, register offset, or pre/post-indexed immediate writeback.
- Scaled register offsets map naturally to array indexing; post-indexing maps naturally to pointer traversal.
- Natural alignment is preferred, although ordinary unaligned accesses may be permitted depending on the access type and memory attributes.
- ELF/process layout is operating-system and toolchain context, not part of the A64 addressing-mode definition.

Self-Test 1/2

1. What does the bracket notation in `ldr x0, [x1]` mean?
2. How many bytes are read by `ldrb`, `ldrh`, `ldr w0, [...]`, and `ldr x0, [...]`?
3. What is the difference between `ldrb w0, [x1]` and `ldrsb x0, [x1]`?
4. For `ldr x0, [x1, #16]`, what is the effective address and what happens to `x1`?
5. Why is `ldr x0, [x1, x2, lsl #3]` suitable for a 64-bit array?

Self-Test 2/2

6. What changes in `x1` after `ldr x0, [x1, #8]!`?
7. What changes in `x1` after `ldr x0, [x1], #8`?
8. Why is `ldr x0, [x1, x2]!` not a valid example of A64 pre-indexed register addressing?
9. What does natural alignment mean for an 8-byte object?
10. Why should ELF sections such as `.text` and `.data` not be described as A64 addressing modes?

Next Topic

Branching and Iteration

Topic 7 uses the register and memory operations developed so far to build control-flow structures.

- Comparisons and NZCV condition flags
- Conditional and unconditional branches
- Translation of `if` and `if-else`
- `while` and counted loops
- Array and string traversal with branches